

GUIDE TECHNIQUE & TUTORIEL

Mon PF App

Application mobile de gestion des commandes et livraisons d'un restaurant

Champ	Valeur
Etudiante	Yasmine Hajji
Filiere	Developpement d'applications
Contexte	Projet de fin d'etudes
Version du guide	13 juin 2026
Plateformes cibles	Android en priorite, Windows en preparation

Objectif: expliquer ce qui a ete construit, pourquoi les choix techniques ont ete faits, comment les pieces se connectent, et comment tester/presenter l'application.

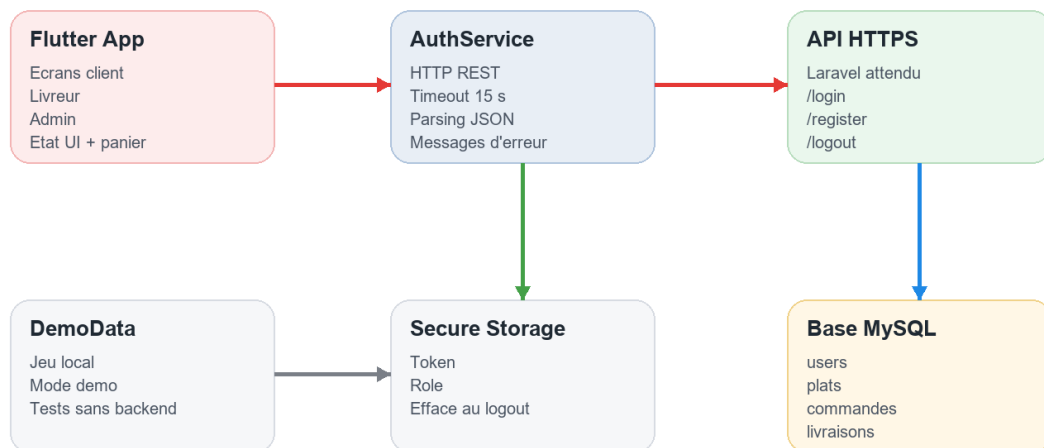
1. Vue d'ensemble

Mon PF App digitalise le parcours d'un restaurant: le client consulte le menu, ajoute des plats au panier, valide une commande et suit la livraison. Les espaces livreur et administrateur montrent le pilotage operationnel.

Le mode demonstration est actif par default pour tester toute l'interface sans backend. Quand l'API Laravel/MySQL sera prete, l'application pourra basculer vers le vrai serveur avec les dart-defines Flutter.

Acteur	Responsabilite	Ecrans
Client	Se connecter, parcourir le menu, gerer le panier, suivre la commande.	Login, Accueil, Menu, Panier, Suivi, Profil
Livreur	Voir les livraisons disponibles, accepter/refuser, suivre ses indicateurs.	Tableau de bord livreur
Administrateur	Observer ventes, commandes, clients, livreurs et alertes stock.	Tableau de bord administration
Serveur	Representable dans le flux operationnel; peut devenir un espace dedie.	Extension future

Architecture globale



Le meme frontend peut fonctionner en mode demo ou contre une vraie API REST securisee.

Figure 1 - Architecture cible et mode demo.

2. Stack technique

Technologie	Ou	Pourquoi
Flutter / Dart	mon_pfapp/lib	Un codebase pour Android, Windows, web de test et futures plateformes.
Material 3	MyApp + shared/widgets	Composants propres et theme coherent.

HTTP REST	AuthService	Integration simple avec Laravel.
flutter_secure_storage	Storage	Protection locale du token et du role.
Laravel + MySQL	Backend attendu	Modele classique pour users, plats, commandes, livraisons.
Gradle / Android SDK	android/	Build APK debug/release.

3. Repository et architecture Flutter

Organisation du repository

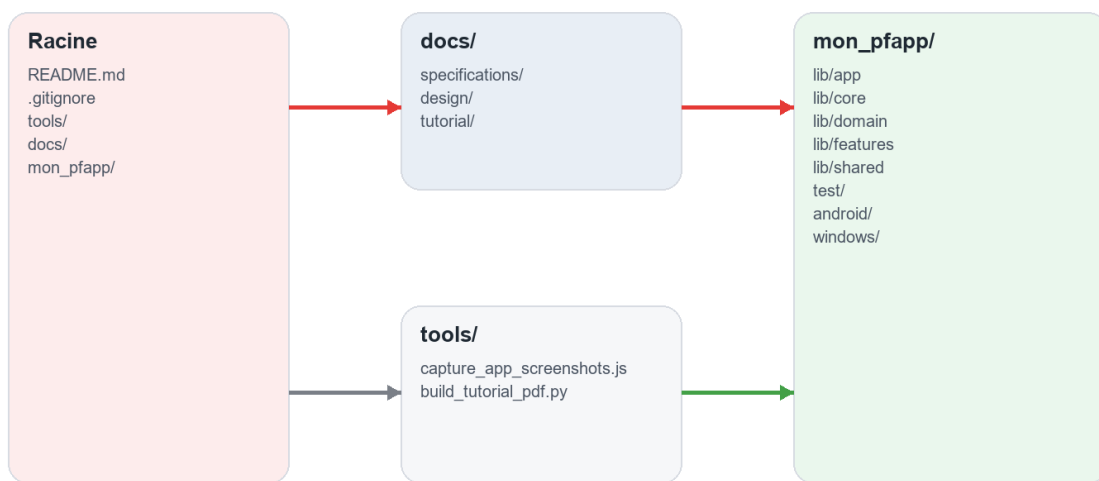


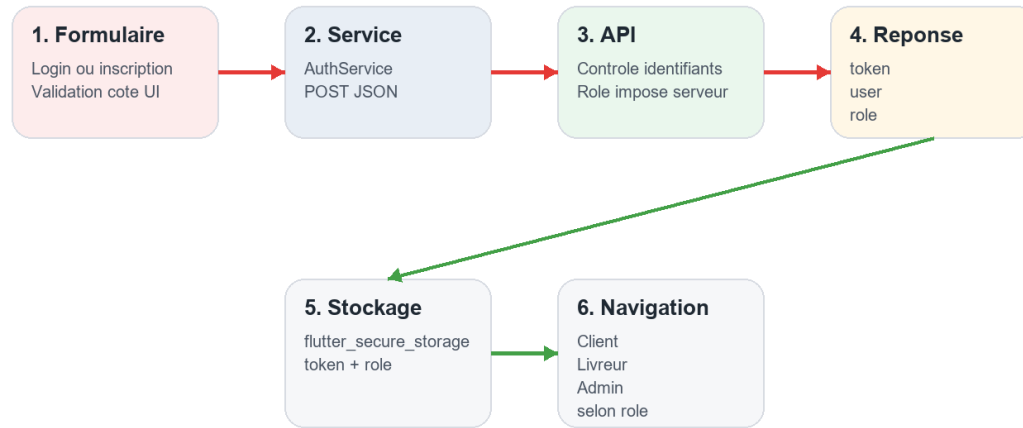
Figure 2 - Organisation professionnelle du depot.

Zone	Role
docs/specifications	Cahier de charge et analyse/conception originaux.
docs/design	Reference visuelle de la maquette.
docs/tutorial	PDF, DOCX, diagrammes et captures.
tools	Scripts de capture et generation du guide.
lib/app	Shell applicatif, navigation et panier de demonstration.
lib/core	Configuration API, secure storage, validateurs.
lib/features	Ecrans par domaine: auth, client, operations.

MonPfApp possede l'etat de demonstration: ecran courant, utilisateur connecte et panier. Les ecrans recoivent des callbacks, ce qui garde la navigation lisible pour la soutenance.

4. Authentification et securite

Flux d'authentification



Important: le formulaire public ne choisit jamais le role; le backend retourne le role autorise.

Figure 3 - Connexion, role et stockage securise.

Risque initial	Correction	Impact
API locale/en clair	API_BASE_URL par dart-define et HTTPS par default.	Configuration propre pour prod.
Role choisi publiquement	Inscription publique toujours client; roles serveur.	Evite l'auto-promotion.
Token stockage simple	flutter_secure_storage.	Jeton mieux protege.
Serveur lent	Timeout 15 s + messages d'erreur.	UI plus robuste.
Async apres fermeture	Checks mounted.	Evite les setState invalides.

5. Modele de donnees

Modele de donnees cote Flutter

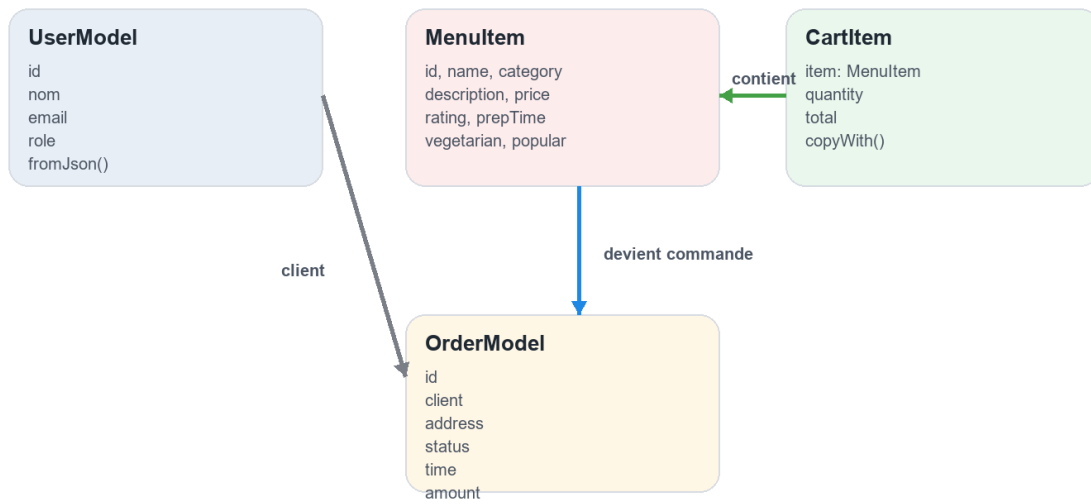


Figure 4 - Classes métier cote Flutter.

Classe	Champs	Utilisation
UserModel	id, nom, email, role	Utilisateur authentifie et navigation selon role.
MenuItem	id, name, category, price, rating, prepTime	Afficher plats, filtrer et ajouter au panier.
CartItem	item, quantity, total	Calculer le total ligne et gerer quantites.
OrderModel	id, client, address, status, time, amount	Afficher les commandes operations.

6. Algorithmes et regles metier

L'application utilise des algorithmes applicatifs simples et defendables: panier, total, routage d'ecran, controle token/role, decode JSON, timeout et progression commande.

Mecanisme	Comment ca marche	Pourquoi
Ajout panier	Recherche par id; ajoute un CartItem ou incremente la quantite.	Evite les doublons.
Quantite	Delta +1/-1 puis clamp entre 1 et 99.	Pas de quantite negative.
Total	CartItem.total = prix * quantite; panier additionne sous-total/livraison/remise.	Facture claire.
Navigation	switch sur _screen.	Simple a expliquer.
Auth API	POST JSON, controle status, token+user requis, timeout.	Robustesse reseau.
Role	Client a l'inscription, role sensible via backend.	Securite.

Cycle de vie d'une commande

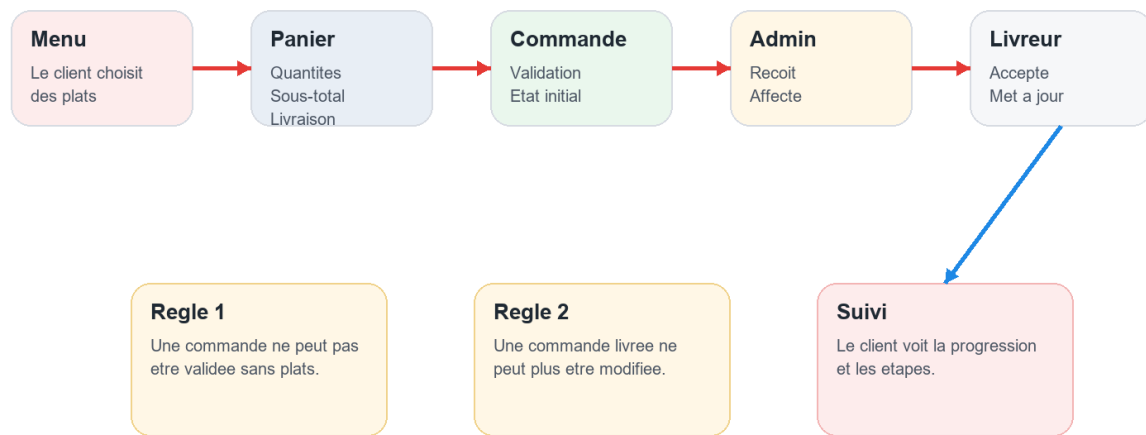


Figure 5 - Cycle de vie et règles d'une commande.

7. Captures de l'application

Les captures viennent de la version web release Flutter. Elles montrent que le parcours est testable même sans backend.

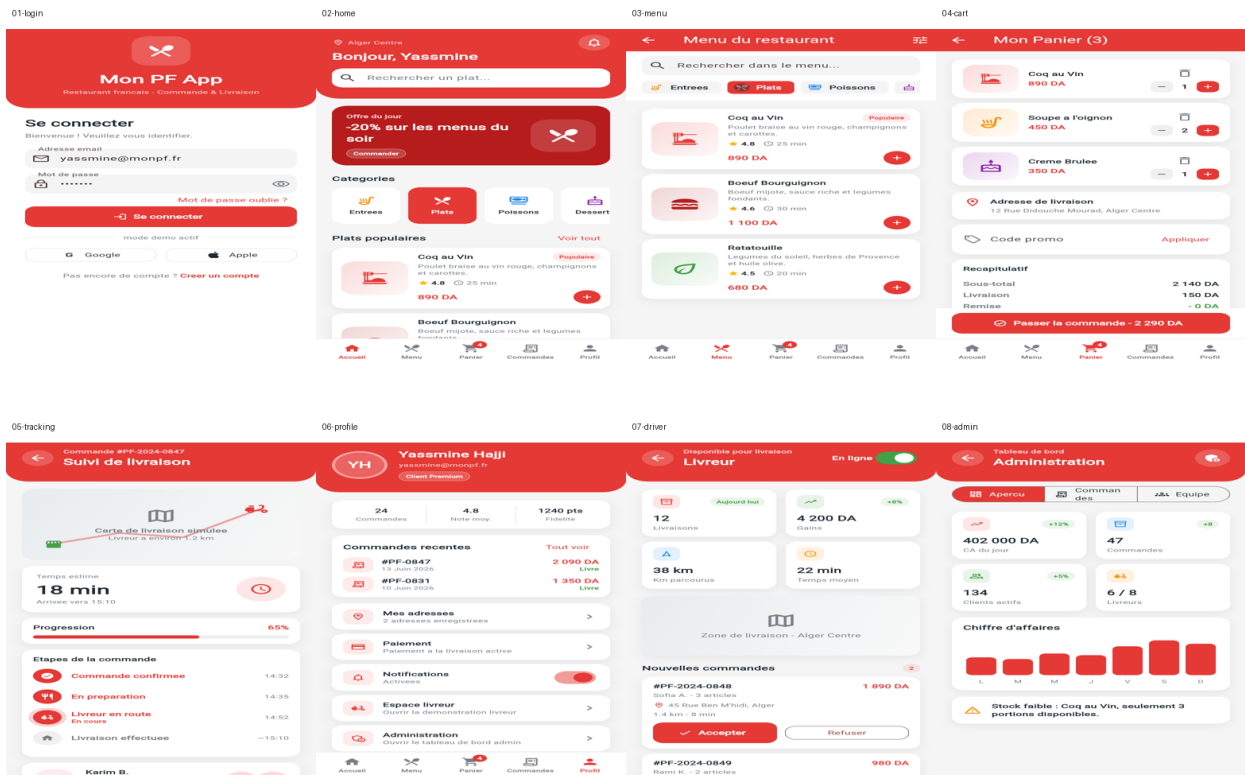
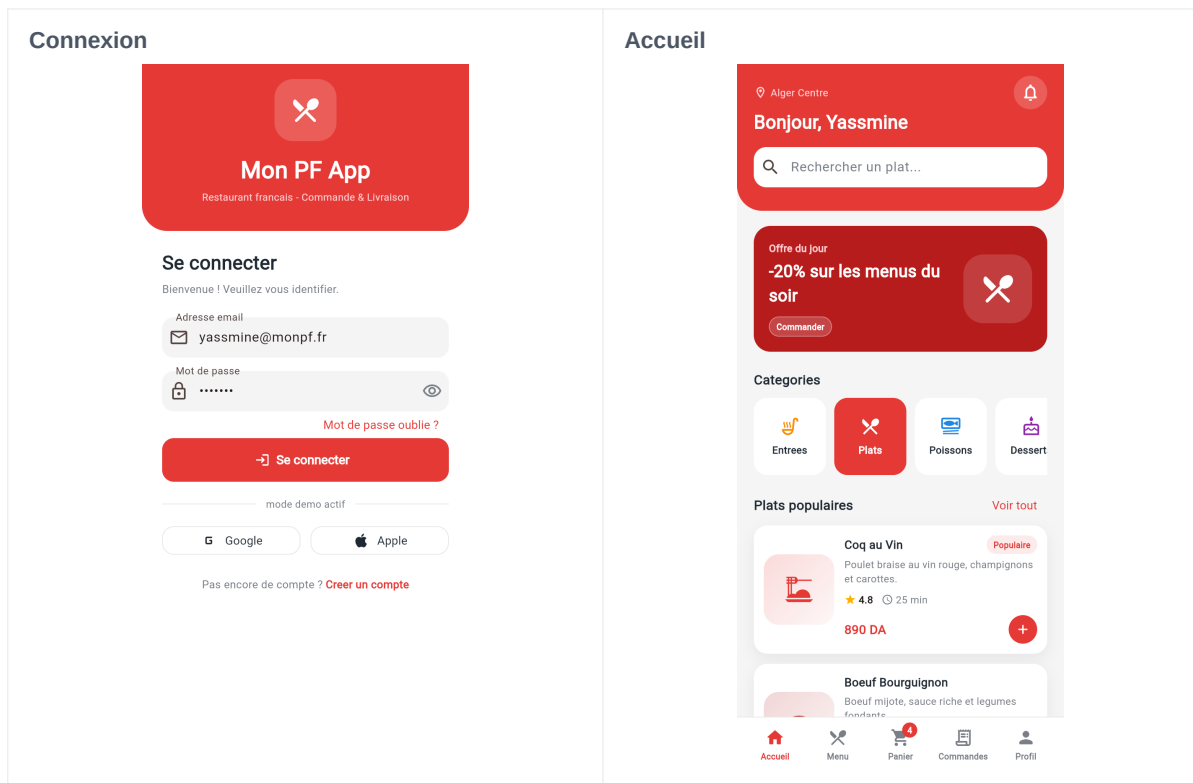
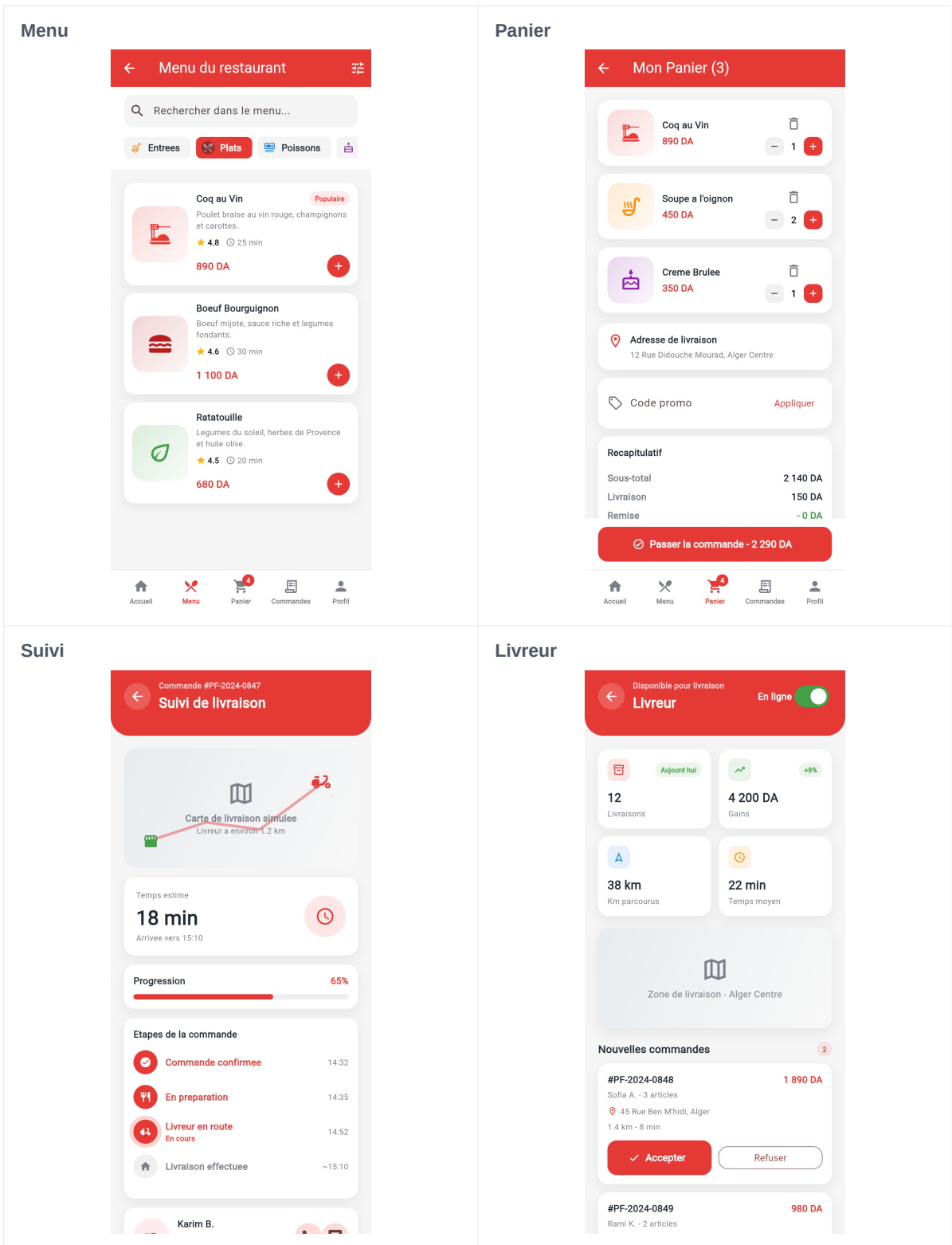


Figure 6 - Vue rapide des écrans principaux.

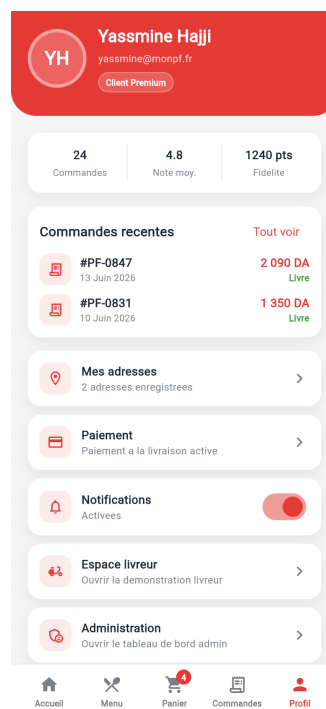




Admin



Profil



8. Tests, builds et production

Controle	Commande / resultat	Etat
Analyse Dart	dart analyze	OK
Tests Flutter	flutter test	5 tests OK
Web release	flutter build web --release --dart-define=DEMO_MODE=true	OK
Android debug	flutter build apk --debug	OK
Android release signe	flutter build apk --release	OK, APK genere
Windows release	flutter build windows --release	Bloque par ATL manquant

Windows: installer le composant Visual Studio 'C++ ATL for latest v142 build tools' pour resoudre l'erreur atistr.h de flutter_secure_storage_windows.

Commandes utiles

```
cd mon_pfapp
flutter pub get
flutter test
flutter run
```

```
flutter run -d chrome --web-port 57321
```

```
flutter run --dart-define=DEMO_MODE=false
--dart-define=API_BASE_URL=https://votre-domaine/api
```

```
adb install -r build/app/outputs/flutter-apk/app-release.apk
```

9. Reste a faire pour une vraie production

- Brancher les routes Laravel/MySQL finales: auth, menu, commandes, affectation livreur, suivi.
- Ajouter middleware backend pour les roles et permissions.
- Remplacer progressivement DemoData par des appels API.
- Ajouter notification temps reel ou polling pour le suivi livraison.
- Garder les cle de signature Android hors Git.
- Installer ATL pour la version Windows.
- Ajouter CI/CD GitHub Actions pour analyse, tests et builds.

10. Script de presentation au jury

1. Presenter la problematique: erreurs, retards et manque de visibilite dans une gestion manuelle.
2. Presenter la solution: app Flutter connectant client, restaurant, livreur et admin.
3. Demontrer le parcours client: login, menu, panier, suivi.
4. Demontrer les espaces operations: livreur puis admin.
5. Expliquer les choix securite: HTTPS/env, secure storage, roles serveur, timeouts.
6. Conclure sur les tests, l'APK Android et les limites honnetes.

Questions probables du jury

Question	Reponse courte
Pourquoi Flutter ?	Pour livrer rapidement Android et garder une base compatible web/desktop.
Pourquoi secure storage ?	Un token d'authentification ne doit pas rester dans un stockage simple.
Pourquoi supprimer le choix du role ?	Un utilisateur public ne doit jamais pouvoir se creer admin ou livreur.
Pourquoi le mode demo ?	Pour garantir une soutenance testable meme si le backend n'est pas encore branche.
Pourquoi Windows est bloque ?	Il manque le composant ATL de Visual Studio, requis par le plugin secure storage Windows.